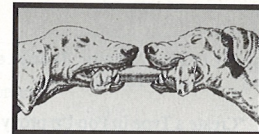


PROGRAMMER'S CHALLENGE

By Bob Boonstra, Westford, MA



HEX

This month features another round-robin tournament competition, this time with the game of Hex. You might have encountered Hex through one of Marvin Gardner's columns in Scientific American. The game is played on an NxN rhombus of hexagons, with one player attempting to occupy an unbroken chain of hexagons connecting the top and the bottom, and the other player the left with the right. Players alternate occupying hexagons, and the first to complete an unbroken chain is the winner. The prototype for the code you should write is:

```
Boolean /*legalMove*/ Hex (  
    long boardSize, /* number of rows/columns in the game board */  
    long oppRow, /* row where opponent last moved, 0 .. boardSize-1 */  
    long oppCol, /* column where opponent last moved, 0 .. boardSize-1 */  
    long *moveRow, /* return your move - row, 0 .. boardSize-1 */  
    long *moveCol, /* return your move - column, 0 .. boardSize-1 */  
    void *privStorage, /* preallocated storage for your use */  
    Boolean newGame, /* TRUE if this is your first move in a new game */  
    Boolean playFirst /* TRUE if you play first (vertically) */  
);
```

For your first move, **Hex** will be called with **newGame** set to **TRUE**. The size of the board, a number between 8 and 64, inclusive, will be provided in **boardSize**. If **playFirst** is **TRUE**, your objective is to form a vertical connection across the rows of the board, and you make the first move. Otherwise you are playing second and the objective is to form a horizontal connection across the columns. Your code and the code of an opponent will alternate play. Your opponent's move will be provided in (**oppRow**, **oppCol**), which will be set to (-1, -1) if you are moving first. When your code is called, you should evaluate your opponent's move, calculate your own move, and store it in (***moveRow**, ***moveCol**). If for any reason you believe your opponent has moved into an occupied hexagon, **Hex** should return a **legalMove** value of **FALSE**, otherwise it should return **TRUE**.

A key to winning at Hex is to form "2-bridges" of hexagons that can be connected regardless of what move your opponent makes. For example, examine this 9x9 board, with the horizontal ("H") player to move:

```
0 1 2 3 4 5 6 7 8  
0 . . . . . . . .  
1 . . . . . . . V  
2 . . V . . . . .  
3 . . . V . . . .  
4 . . . . V . . .  
5 . . V H H H V .  
6 . . . . . H . .  
7 . H . V . H . .  
8 . H . . . . . .
```

In this example, V can force a win. H can block one of the (row,col) positions (6,2) or (6,5), but not both. If, for example, H occupies (6,2), then V can occupy (6,5), with a guaranteed "2-bridge" connection to (7,3) via (6,4) or (7,4). A similar bridge exists if H occupies (6,5). V is therefore unstoppable from this position.

For some additional strategy, see <http://www.daimi.aau.dk/~tusk/pbmserv/hex/hex.faq.html>, or <http://www.cs.cmu.edu/People/hde/hex/hexfaq>. There is a "proof" that the first player can always win, but the proof is a nonconstructive proof by counterexample that does not divulge the winning strategy.

Your code will be provided with 1MB of preallocated storage pointed to by **privStorage**. This storage will be preinitialized to zero before your first move and will persist between moves. You should not allocate any additional dynamic storage beyond that provided by **privStorage**. Small amounts of static storage are permitted.

THE RULES

Here's how it works: each month we present a new programming challenge. First, write some code that solves the challenge. Second, optimize your code (a lot). Then, submit your solution to MacTech Magazine. We choose a winner based on code correctness, speed, size, and elegance (in that order of importance) as well as the submission date. In the event of multiple equally desirable solutions, we'll choose one winner (with honorable mention, but no prize, given to the runner up). The prize for each month's best solution is a \$100 credit for Developer Depot™.

Unless stated otherwise in the problem statement, the following rules apply: All solutions must be in ANSI compatible C or C++, or in Pascal. We disqualify entries with any assembly in them (except for challenges specifically stating otherwise.) You may call any Macintosh Toolbox routine (e.g., it doesn't matter if you use **NewPtr** instead of **malloc**). We compile all entries into native PowerPC code with compiler options set to enable all available speed optimizations. The development environment to be used for selecting the winner will be stated in the problem. **Limit your code to 60 characters per line** or compress and binhex the solution; this helps with e-mail gateways and page layout.

We publish the solution and winners for each month's Programmer's Challenge three months later. All submissions must be received by the 1st day of the month printed on the front cover of this issue.

You can get a head start on the Challenge by reading the Programmer's Challenge mailing list. It will be posted to the list on or before the 12th of the preceding month. To join, send an email to listserv@listmail.xplain.com with the subject "subscribe challenge-A".

Mark solutions "Attn: Programmer's Challenge Solution" and send it by e-mail to one of the Programmer's Challenge addresses in the "How to Communicate With Us" section on page 2 of this issue. Include the solution, all related files, and your contact info.

MacTech Magazine reserves the right to publish any solution entered in the Programmer's Challenge. Authors grant MacTech Magazine the exclusive right to publish entries without limitation upon submission of each entry. Authors retain copyrights for the code.

The Challenge will be scored by a tournament where each entry plays against each other entry twice for each of a number of board sizes, once playing first and once playing second. Another fair tournament schedule might be used if a large number of entries are received. For each game, the winner will earn game points for the victory, minus an amount based on the execution time used to compute the moves, as follows:

$$\text{game points} = 10 - (\text{execution time in seconds}) / \text{boardSize}$$

The loser will earn zero points and will not be penalized for the execution time expended in defeat. The player with the most total points for all games in the tournament will be the winner.

This will be a native PowerPC Challenge, using the latest CodeWarrior environment. Solutions may be coded in C, C++, or Pascal.

THREE MONTHS AGO WINNER

The December Tangram Challenge was a difficult one, so difficult that no entries were submitted by the Challenge deadline. Three people (Alan Hart, Ernst Munter, and George Warner) responded with entries after I notified the Programmer's Challenge mailing list of a one-week extension, but none of those entries passed my test cases. As a result, there is no winner for the Challenge this month.

The problem was to reassemble a two-dimensional shape that has been cut into a number of smaller shapes. Each piece was to be transformed to its correct position by optionally flipping it about a vertical axis, rotating it, and translating it, in that order. A simple tangram can be created by curving a square into five right triangles of three different sizes, a smaller square, and a rhomboid. The problem statement also allowed for zero or more holes in the assembled shape, a fact which complicated the problem significantly. The submitted solutions all had problems with shapes that did not include holes, and I was unable to select one as being more correct than the rest. I have awarded 4 contest points to each of the entrants for their efforts, and selected one of them for publication.

Dave Bayer wrote to point out that the Tangram Challenge is theoretically as well as practically difficult, in that it is one of a class of problems called "NP-hard". The conjecture is that these problems cannot be solved in time that is a polynomial function of the problem size, and the problems in this class are equivalent in the sense that a polynomial time solution for one of them would provide a solution for the others. The best known problem in this class is the traveling salesperson problem.

To demonstrate that the Tangram Challenge is NP-hard, Dave transforms another NP-hard problem, the Partition Problem, into a tangram. The partition problem is to determine, given a sequence of integers $i^1, i^2, \dots, i^k$, whether there exists a subset whose sum is exactly half of the sum of the entire sequence. For example, given the sequence 4,4,5,7,9,11,14,14,14,18, which sums to 100, the Partition Problem would be to find a subset (e.g., 4,7,11,14,14) that sums

Name Your Poison

Java OpenDoc
(ODF)
MPW PowerPlant
Roaster
CodeWarrior
Procedural C
SYMANTEC
neoAccess
TCL
OOFILE
Tools Plus

AppMaker supports most popular languages and frameworks. Just point and click to design your user interface, then let AppMaker create resources and generate "human, professional quality code" to implement your design.

Use AppMaker as a prototyping and productivity tool or use it as a learning tool for Mac programming techniques or for new environments such as OpenDoc, Java, or PowerPlant.

AppMaker is just \$299 and includes a one-year subscription on CD.

**B • O • W • E • R • S
Development**

603-863-0945

bowersdev@aol.com

<http://members.aol.com/bowersdev>

Your code will be timed with a sequence of rule/tape pairs, and the solution that completes execution of the inputs correctly in the shortest total time will be the winner. Half of the tests will use a "universal" Turing Machine, where the rules describe a general Turing Machine interpreter, and the input tape contains another Turing Machine program. The time executed by the **ReportMove** callback will be included in your solution time. The callback will look something like the following.

```
static void ReportMove(long outputSymbol, long newState,
MoveDir moveDirection)
{
    if ((gTapePos>=0) && (gTapePos<gTapeLen))
        gTape[gTapePos] = outputSymbol;
    gTapePos += moveDirection;
    gState = newState;
}
```

You may not modify the input rules pointed to by **theRules**, but you may allocate reasonable additional storage if you would like to preprocess the rules in some way, provided you deallocate the storage before returning.

For those of you that would like additional information on Turing Machines, you can refer to almost any textbook on automata theory or the theory of computation. My personal favorite is *Computation, Finite and Infinite Machines*, by Marvin Minsky, © 1976 by Prentice-Hall.

This will be a native PowerPC Challenge, using the latest CodeWarrior environment. Solutions may be coded in C, C++, or Pascal.

THREE MONTHS AGO WINNER

Congratulations to **Gregory Cooper** for narrowly defeating the second place entry by **Jeff Mallett** in the March Hex Challenge. Hex is played on an NxN rhombus of hexagons, with players alternately occupying hexagons, one playing vertically and the other horizontally, each trying to complete an unbroken chain before the other does. The Challenge awarded 10 points for each victory in a round-robin tournament, with the score reduced based on the execution time of the winning solution.

The third-place solution by Eric Hangstefer used a purely offensive strategy, building bridges in an attempt to force the opponent to play defense. The other two solutions applied both offensive and defensive techniques, and between them won all of the games in the tournament. Gregory's solution plays defense whenever it believes that its opponent is closer to a solution than he is. To accomplish this, he maintains a Path data structure for himself and for his opponent identifying what he believes to be the best (shortest) path across the board for each player. A path score is maintained by counting the number of hexagons along the path that still need to be occupied to complete an unbroken chain (excluding "two-bridge" positions where a connection is guaranteed). One key to understanding the winning solution is the **ExtendPathToEdge** routine, used by the **BestPathAcrossBoard** routine to find a short route to each edge from a central hexagon.

The tournament consisted of three test cases, with hex boards of size 8, 10, and 18, with each solution competing against each other twice, once playing first and once playing second. Gregory's

solution and Jeff's solution competed against one another six times, and, interestingly enough, in all but one of those contests, the solution playing second (i.e., horizontal) won the game. In most of the games, Gregory's and Jeff's solutions required comparable execution time, except in two instances where Gregory's solution took very long to make its final move. This behavior accounted for the fact that the two solutions were very close in cumulative points, even though Gregory won two more games. By virtue of having the highest point total, the most victories, and (as a tie-breaker) the smallest code, Gregory's solution is the winner.

The final position in the 10x10 game where Gregory played the vertical orientation and won is reproduced below for your enjoyment. Note the vertical player's focus on a single path by comparison with the more scattered hexagons occupied by the horizontal player.

```
0 1 2 3 4 5 6 7 8 9
0 - - V H H - - - H
1 - - V V - H - - H V
2 - - H V - - H - V -
3 - - V H H H - - -
4 - - V - - V - V - -
5 - V H H - - - - -
6 - V V H H - V - -
7 - - V V H - - V - -
8 - - H V - H - - -
9 - - V H - - - - -
```

The table below lists for each entry the number of tournament wins, the total tournament points earned, and the code and data sizes. The number in parentheses after the entrant's name is the total number of Challenge points earned in all Challenges to date prior to this one.

Name	Wins	Points	Code	Data
Gregory Cooper (34)	10	80.08	8028	628
Jeff Mallett (64)	8	79.98	8608	392
Eric Hangstefer (2)	0	0.00	5236	88

TOP 20 CONTESTANTS

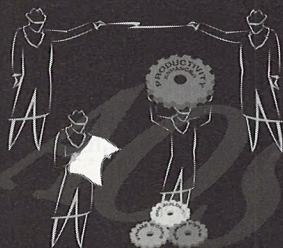
Here are the Top Contestants for the Programmer's Challenge. The numbers below include points awarded over the 24 most recent contests, including points earned by this month's entrants.

Rank	Name	Points	Rank	Name	Points
1.	Munter, Ernst	184	11.	Cutts, Kevin	21
2.	Gregg, Xan	114	12.	Nicolle, Ludovic	21
3.	Larsson, Gustav	67	13.	Picao, Miguel Cruz	21
4.	Cooper, Greg	54	14.	Brown, Jorg	20
5.	Lengyel, Eric	40	15.	Gundrum, Eric	20
6.	Boring, Randy	37	16.	Higgins, Charles	20
7.	Mallett, Jeff	37	17.	Kasparian, Raffi	20
8.	Lewis, Peter	32	18.	Slezak, Ken	20
9.	Antoniewicz, Andy	24	19.	Studer, Thomas	20
10.	Beith, Gary	24	20.	Karsh, Bill	19

SmalltalkAgents®

Integrated Development Environment

for
Macintosh
Windows 95/NT



Check out what's new...

www.SmalltalkAgents.com


1405 Main Street
P.O. Box 371478
Montara, CA 94037-1478 USA

© Quasar Knowledge Systems, Inc. 1997

Windows 95 and Windows NT are Trademarks of Microsoft.

There are three ways to earn points: (1) scoring in the top 5 of any Challenge, (2) being the first person to find a bug in a published winning solution or, (3) being the first person to suggest a Challenge that I use. The points you can win are:

1st place20 points	5th place.....2 points
2nd place.....10 points	finding bug.....2 points
3rd place.....7 points	suggesting Challenge...2 points
4th place.....4 points	

Here is Gregory's winning solution:

Hex_Strategy.c © 1997 Gregory Cooper

```
// *****
// General idea: My strategy tries to find the quickest routes across the board, both for
// itself and for its opponent. If the opponent has a better, shorter route than my
// strategy does, then my strategy makes a defensive move (one that makes it more
// difficult for the opponent to complete his route). Otherwise, it makes an offensive
// move (one which brings its own route closer to completion). It re-traces the routes
// during each turn, looking for necessary modifications and scoring changes.
// *****
```

#include "Storage.h"

// an enumeration for move types

typedef enum

```
{
    // counter-clockwise from right
    right, // 0
    upRightTwoBridge, // 30
    upRight, // 60

```

```
upTwoBridge, // 90
upLeft, // 120
upLeftTwoBridge, // 150
left, // 180
downLeftTwoBridge, // 210
downLeft, // 240
downTwoBridge, // 270
downRight, // 300
downRightTwoBridge, // 330
noDirection // undefined
} Direction;
```

// a data structure for representing potential routes to edges or to other chips

typedef struct Path

```
{
    long row; // row of piece on board
    long col; // column of piece on board
    Direction nextDirection; // direction to next piece
    Direction prevDirection; // direction to prev piece
    Boolean occupied; // location actually taken?
    Boolean center; // origin of path?
    struct Path*next; // next piece in path
    struct Path*prev; // previous piece in path
} Path;
```

// a data type for the playing chips

typedef enum {

```
empty,
blue, // plays first
red // plays second
} Chip;
```

// an enumeration for disposing of unneeded paths

// (used by the BetterPath function and its clients)

typedef enum {

```
leaveBad, // indicates that inferior strategy should be left alone
forgetBad // indicates that inferior strategy should be disposed
} PathAction;
```

// an enumeration for the edges

typedef enum {

```
leftEdge,
rightEdge,
topEdge,
bottomEdge
} Edge;
```

// an enumeration for path disruptions

typedef enum {

```
noDisruption, // move did not block path
inTwoBridge, // move occupied part of a two-bridge
inSpot // move occupied a strategic location
} Disruption;
```

Function Prototypes

```
Path *BestPathAcrossBoard(long boardSize, Chip *theBoard,
    long startRow, long startCol, Chip color);
// tries to find a path across the board through (startRow, startCol)
// PRECONDITIONS: boardSize is the number of rows or columns
// on the board; theBoard stores information about the board;
// (startRow, startCol) indicates a piece of the given color on the board; color indicates
// which player for whom the path is to be found
// POSTCONDITIONS: returns a pointer to the path found, beginning with
// (startRow, startCol), or returns nil if no path exists
```

```
Boolean ExtendPathToEdge(long boardSize, Chip *theBoard,
    Path *path, Edge edge);
// tries to find a path from path to the given edge
// PRECONDITIONS: path is an incomplete path, edge is one of the four edges,
// boardSize is the number of rows or columns on the board, theBoard defines the
// playing board
// POSTCONDITIONS: if a path from the edge exists, it is found and the value true is
// returned; otherwise, all memory associated with the path is freed and it returns false
```

```
void PrioritizeMoves(Edge edge, Direction moveList[12]);
// determines the order in which potential moves in a path should be attempted
// PRECONDITIONS: edge is the edge to which we are trying to move
// POSTCONDITIONS: moveList contains the moves in order from most to
// least direct
```

```
Boolean PossibleMove(long boardSize, Chip *theBoard,
    long row, long col, Path *path, Direction move,
    Chip color);
// determines whether placing a piece in a given location is feasible
```